

# Transformer

## Stock price demo tutorial

### (PyTorch implementation of transformer)

[My Drive/Colab Notebooks/elec6604python/Transformer-stock-price-demo.ipynb](#)

# **Transformer-stock-price-demo.ipynb**

# Oct 7, 2026

#

# This is a demo of an PyTorch implementation of a transformer **time series forecasting example**.

# It uses the previous **20 days of stock price** of the Apple (ASPL) stock to predict the next value.

# The **daily closing price is used** and the model takes the previous 20 trading days as input

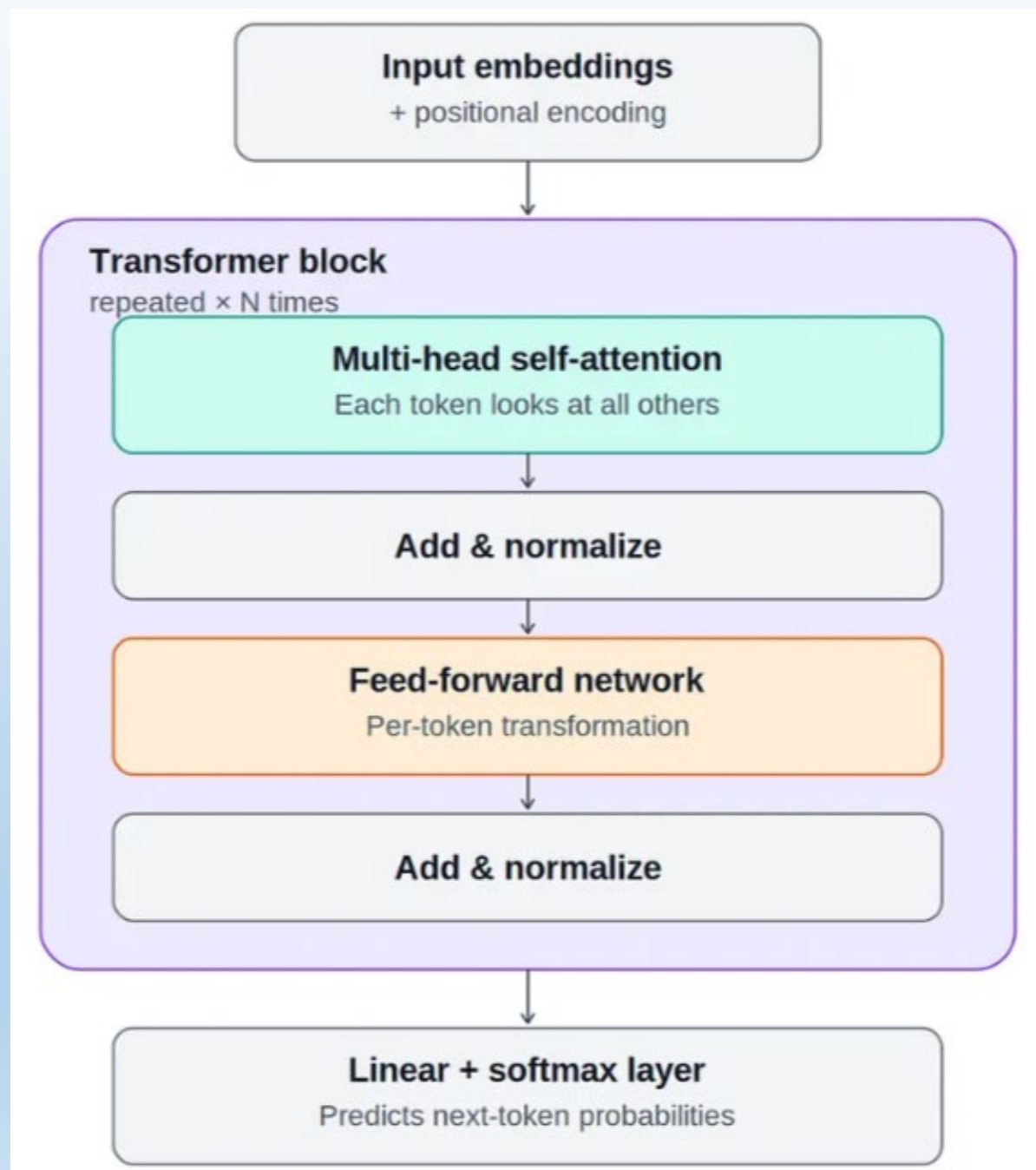
# **to predict the next closing price**, and then recursively forecasts multiple future values.

# The transformer is the machine learning model used for training. **The dataset is aapl\_demo.csv.**

# On the transformer architecture, take note of the manual PyTorch implementation of the

# **scaled dot product and the attention mechanism**. The usual built-in function TransformerEncoder

# or the TransformerEncoderLayer is not used. **Pay attention to the dimension of the matrices.**



20 values

$x_{t-19} \ x_{t-18} \ \dots \ x_t \ \longrightarrow \text{Predict next value}$

```
# =====  
# 1. Configuration  
# =====  
LOOKBACK = 20  
BATCH_SIZE = 16  
NUM_EPOCHS = 300  
LR = 1e-3  
  
D_MODEL = 64  
NHEAD = 4  
NUM_LAYERS = 2  
DIM_FEEDFORWARD = 128  
DROPOUT = 0.1  
  
torch.manual_seed(42)  
np.random.seed(42)
```

Each single value is represented by 64 numbers.

Each attention block has 4 heads.

Two attention blocks.

The feedforward neural network has 128 hidden nodes.  
Hence, the structure is  $64 \rightarrow 128 \rightarrow 64$

```
print("First, to illustrate matrix multiplication:")
```

```
K_demo = np.array([[1.0, 0.0, 0.5, 0.2], [0.0, 1.0, 0.8, 0.4]])
```

```
K_demo_new = np.swapaxes(K_demo, -1, -2)
```

```
print(K_demo)
```

```
print(K_demo_new)
```

```
print("K_demo.shape (2 x 4 matrix) = ", K_demo.shape)
```

```
print("K_demo_new.shape (4 x 2 matrix) = ", K_demo_new.shape)
```

```
result_demo = np.matmul(K_demo,K_demo_new)
```

```
print("result_demo (2 x 2 matrix) = ")
```

```
print(result_demo)
```

First, to illustrate matrix multiplication:

```
[[1.  0.  0.5 0.2]  
 [0.  1.  0.8 0.4]]
```

```
[[1.  0. ]  
 [0.  1. ]
```

```
 [0.5 0.8]
```

```
 [0.2 0.4]]
```

K\_demo.shape (2 x 4 matrix) = (2, 4)

K\_demo\_new.shape (4 x 2 matrix) = (4, 2)

result\_demo (2 x 2 matrix) =

```
[[1.29 0.48]  
 [0.48 1.8 ]]
```

```
print("To illustrate the softmax operation:")
```

```
def softmax(x):
```

```
    exp_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
```

```
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)
```

```
scores = np.array([[2, 1], [1, 3]])
```

```
print("scores simulates the scaled dot-product scores")
```

```
print("scores")
```

```
print(scores)
```

```
weights = softmax(scores)
```

```
print("weights")
```

```
print(weights)
```

```
print("Note that sum of both rows are 1.")
```

To illustrate the softmax operation:

scores simulates the scaled dot-product scores

scores

```
[[2 1]
```

```
 [1 3]]
```

weights

```
[[0.73105858 0.26894142]
```

```
 [0.11920292 0.88079708]]
```

Note that sum of both rows are 1.

```

# =====
# 2. Utility functions
# =====

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

def softmax(x):
    exp_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)

def scaled_dot_product_attention_numpy(Q, K, V):
    d_k = 2
    scores = np.matmul(Q, np.swapaxes(K, -1, -2)) / np.sqrt(d_k)
    weights = softmax(scores)
    return np.matmul(weights, V), weights

```

Each row is softmax()

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

```
# Tiny educational NumPy demo
Q_demo = np.array([[1.0, 0.5, 2.0], [0.0, 3.0, 1.5]])
K_demo = np.array([[2.0, 1.0, 2.5], [ 0.5, 0.2, 0.8]])
V_demo = np.array([[10.0, 1.0, 3.9], [2.0, 20.0, 1.0]])
```

```
print(Q_demo)
print(K_demo)
print(V_demo)
print("Q_demo.shape = ", Q_demo.shape)
print("K_demo.shape = ", K_demo.shape)
print(" ----- ")
output_demo, attention_weights_demo = scaled_dot_product_attention_numpy(Q_demo, K_demo, V_demo)
```

```
print("Tiny demo of NumPy scaled dot-product attention demo output:\n", output_demo)
print("output_demo.shape = ", output_demo.shape)
print(" ----- ")
print("Tiny demo of NumPy attention weights:\n", attention_weights_demo)
print("attention_weights_demo.shape = ", attention_weights_demo.shape)
print(" ----- ")
```

```
[[1.  0.5 2. ]
 [0.  3.  1.5]]
[[2.  1.  2.5]
 [0.5 0.2 0.8]]
[[10.  1.  3.9]
 [ 2.  20.  1. ]]
Q_demo.shape = (2, 3)
K_demo.shape = (2, 3)
```

(2 x 3) matrices

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

```
-----
Tiny demo of NumPy scaled dot-product attention demo output:
[[9.81576139 1.4375667  3.8332135 ]
 [9.7655428  1.55683586 3.81500926]]
output_demo.shape = (2, 3)
-----
Tiny demo of NumPy attention weights:
[[0.97697017 0.02302983]
 [0.97069285 0.02930715]]
attention_weights_demo.shape = (2, 2)
-----
```

```

# =====
# 3. Load dataset
# =====
df = pd.read_csv("/content/drive/My Drive/Colab Notebooks/elec6604python/aapl_demo.csv")
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date").reset_index(drop=True)

series = df["Close"].astype("float32").values

print("Number of rows = ", len(df["Date"]))
print("-----")
print("To display the first five rows in the dataset")

print(df.head())
print("Rows:", len(df))

```

Number of rows = 68

-----  
 To display the first five rows in the dataset

|   | Date       | Open       | High       | Low        | Close      | Adj Close \ |
|---|------------|------------|------------|------------|------------|-------------|
| 0 | 2020-12-07 | 122.309998 | 124.570000 | 122.250000 | 123.750000 | 121.447411  |
| 1 | 2020-12-08 | 124.370003 | 124.980003 | 123.089996 | 124.379997 | 122.065681  |
| 2 | 2020-12-09 | 124.529999 | 125.949997 | 121.000000 | 121.779999 | 119.514076  |
| 3 | 2020-12-10 | 120.500000 | 123.870003 | 120.150002 | 123.239998 | 120.946907  |
| 4 | 2020-12-11 | 122.430000 | 122.760002 | 120.550003 | 122.410004 | 120.132362  |

```
plt.figure(figsize=(12, 4))
plt.plot(df["Date"], series)
plt.title("AAPL Daily Closing Price")
plt.xlabel("Date")
plt.ylabel("Close Price")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



```
# =====  
# 4. Train-test split and scaling  
# =====  
train_ratio = 0.8  
split_idx = int(len(series) * train_ratio)  
  
train_series = series[:split_idx]  
test_series = series[split_idx:]  
  
print("split_idx = ", split_idx)  
print("train_series has items = ", len(train_series))  
print("test_series has items = ", len(test_series))  
print("-----")  
  
scaler = MinMaxScaler(feature_range=(0, 1))  
train_scaled = scaler.fit_transform(train_series.reshape(-1, 1)).flatten()  
test_scaled = scaler.transform(test_series.reshape(-1, 1)).flatten()
```

```
print("Train size:", len(train_series))  
print("Test size:", len(test_series))
```

```
split_idx = 54  
train_series has items = 54  
test_series has items = 14  
-----  
Train size: 54  
Test size: 14
```

```

# =====
# 5. Create sliding windows
# =====
def create_sequences(data, lookback):
    X, y = [], []
    for i in range(len(data) - lookback):
        X.append(data[i:i + lookback])
        y.append(data[i + lookback])

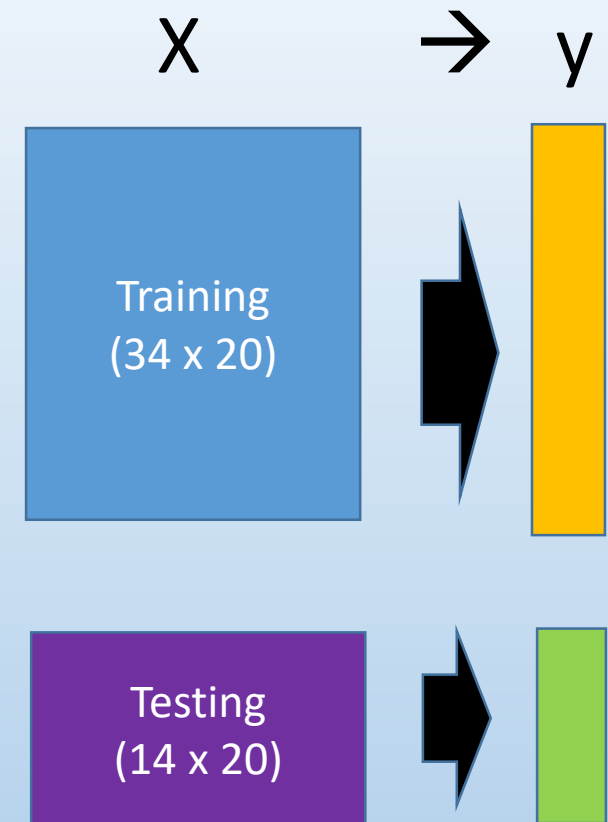
    X = np.array(X, dtype=np.float32).reshape(-1, lookback, 1)
    y = np.array(y, dtype=np.float32).reshape(-1, 1)
    return torch.tensor(X), torch.tensor(y)

X_train, y_train = create_sequences(train_scaled, LOOKBACK)

test_with_context = np.concatenate([train_scaled[-LOOKBACK:], test_scaled])
X_test, y_test = create_sequences(test_with_context, LOOKBACK)

print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
print("X_test :", X_test.shape)
print("y_test :", y_test.shape)
print("-----")
print("X_train and X_test are input sequences of 20 values).")

```



```

X_train: torch.Size([34, 20, 1])
y_train: torch.Size([34, 1])
X_test : torch.Size([14, 20, 1])
y_test : torch.Size([14, 1])

```

-----  
X\_train and X\_test are input sequences of 20 values).

```

# =====
# 6. Positional encoding
# =====
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float32).unsqueeze(1)

        div_term = torch.exp(
            torch.arange(0, d_model, 2, dtype=torch.float32)
            * (-math.log(10000.0) / d_model)
        )

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        pe = pe.unsqueeze(0) # (1, max_len, d_model)
        self.register_buffer("pe", pe)

    def forward(self, x):
        seq_len = x.size(1)
        return x + self.pe[:, :seq_len, :]

```

$$X_{\text{new}} = X + \text{pe}(X)$$

$$= \begin{matrix} \boxed{\begin{matrix} X \\ (34 \times 20) \end{matrix}} + \boxed{\begin{matrix} \text{pe}(X) \\ (34 \times 20) \end{matrix}} \end{matrix}$$

```
# =====  
# 7. Manual scaled dot-product attention in PyTorch  
# =====
```

```
class ScaledDotProductAttention(nn.Module):
```

```
    def __init__(self, dropout=0.0):
```

```
        super().__init__()
```

```
        self.dropout = nn.Dropout(dropout)
```

```
    def forward(self, Q, K, V, mask=None):
```

```
        # Q, K, V: (batch, heads, seq_len, d_k)
```

```
        d_k = Q.size(-1)
```

```
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
```

```
        if mask is not None:
```

```
            # mask should be broadcastable to (batch, heads, seq_len, seq_len)
```

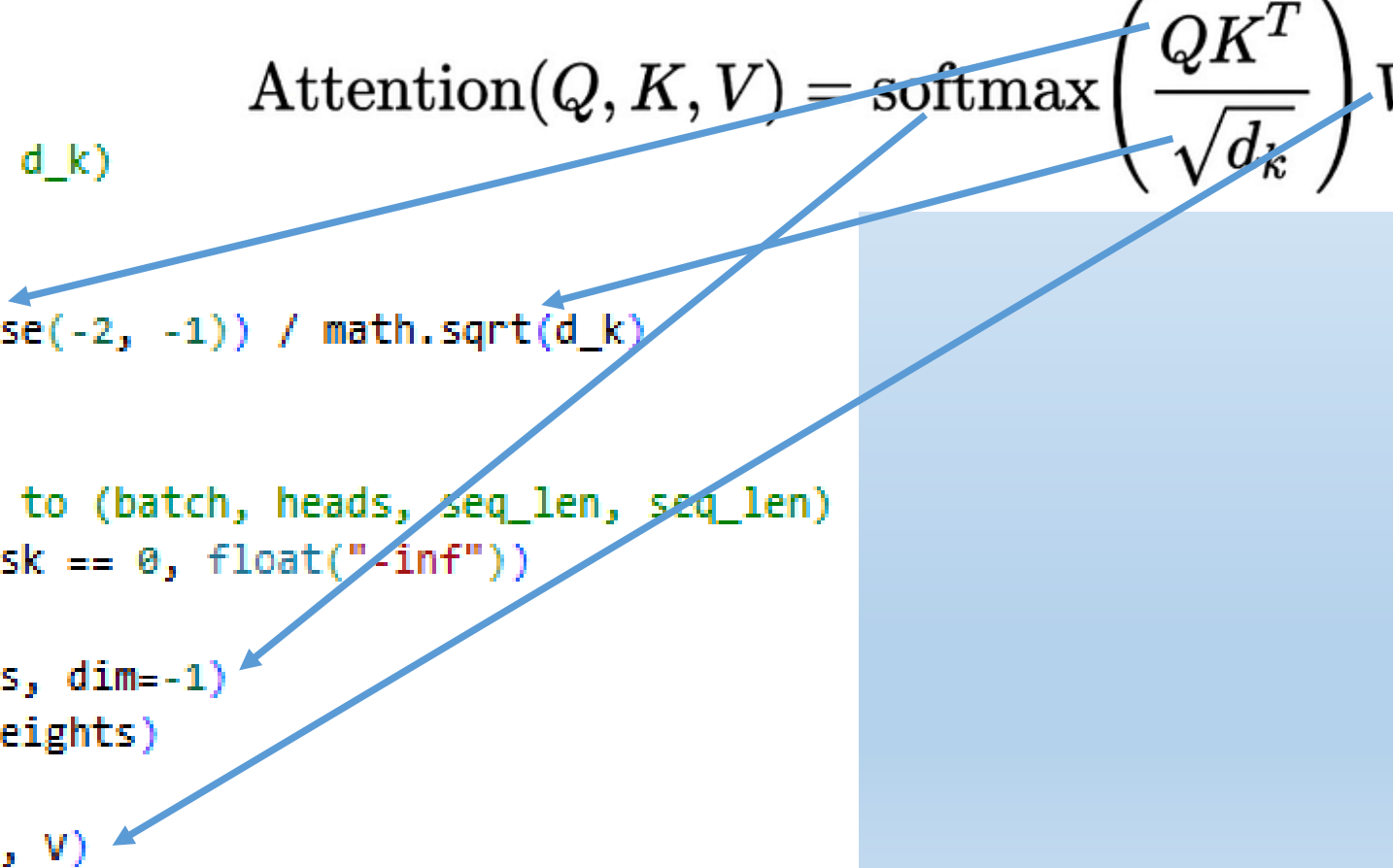
```
            scores = scores.masked_fill(mask == 0, float("-inf"))
```

```
        attn_weights = torch.softmax(scores, dim=-1)
```

```
        attn_weights = self.dropout(attn_weights)
```

```
        output = torch.matmul(attn_weights, V)
```

```
        return output, attn_weights
```

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$


```

# =====
# 8. Manual multi-head self-attention
# =====
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model, nhead, dropout=0.1):
        super().__init__()

        assert d_model % nhead == 0, "d_model must be divisible by nhead"

        self.d_model = d_model
        self.nhead = nhead
        self.d_k = d_model // nhead

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

        self.attention = ScaledDotProductAttention(dropout=dropout)

```

$d\_model = 64$

$nhead = 4$

$d\_k = 16$

The Q,K,V,O are (64x64).

```
def forward(self, x, mask=None):  
    # x: (batch, seq_len, d_model)  
    batch_size, seq_len, _ = x.shape
```

```
    Q = self.W_q(x)  
    K = self.W_k(x)  
    V = self.W_v(x)
```

```
    # reshape to (batch, heads, seq_len, d_k)
```

```
    Q = Q.view(batch_size, seq_len, self.nhead, self.d_k).transpose(1, 2)
```

```
    K = K.view(batch_size, seq_len, self.nhead, self.d_k).transpose(1, 2)
```

```
    V = V.view(batch_size, seq_len, self.nhead, self.d_k).transpose(1, 2)
```

16      20              4              16

```
    attn_output, attn_weights = self.attention(Q, K, V, mask=mask)
```

```
    # merge heads back: (batch, seq_len, d_model)
```

```
    attn_output = attn_output.transpose(1, 2).contiguous().view(  
        batch_size, seq_len, self.d_model
```

```
)
```

```
    output = self.W_o(attn_output)
```

```
    return output, attn_weights
```

Multi-head self-attention is implemented by projecting the input into Q, K, and V, splitting the embedding dimension into multiple heads, applying scaled dot-product attention independently per head, concatenating the head outputs, and then applying an output projection.

Explanations to follow !

```
# =====
# 9. Feedforward block
# =====

class PositionwiseFeedForward(nn.Module):
    def __init__(self, d_model, dim_feedforward, dropout=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, dim_feedforward),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(dim_feedforward, d_model)
        )

    def forward(self, x):
        return self.net(x)
```

64 → 128 → 64

```

# =====
# 10. Manual transformer encoder block
# =====
class TransformerBlock(nn.Module):
    def __init__(self, d_model, nhead, dim_feedforward, dropout=0.1):
        super().__init__()

        self.self_attn = MultiHeadSelfAttention(
            d_model=d_model,
            nhead=nhead,
            dropout=dropout
        )
        self.norm1 = nn.LayerNorm(d_model)
        self.dropout1 = nn.Dropout(dropout)

        self.ffn = PositionwiseFeedForward(
            d_model=d_model,
            dim_feedforward=dim_feedforward,
            dropout=dropout
        )
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout2 = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        attn_output, attn_weights = self.self_attn(x, mask=mask)
        x = self.norm1(x + self.dropout1(attn_output))
        ffn_output = self.ffn(x)
        x = self.norm2(x + self.dropout2(ffn_output))
        return x, attn_weights

```

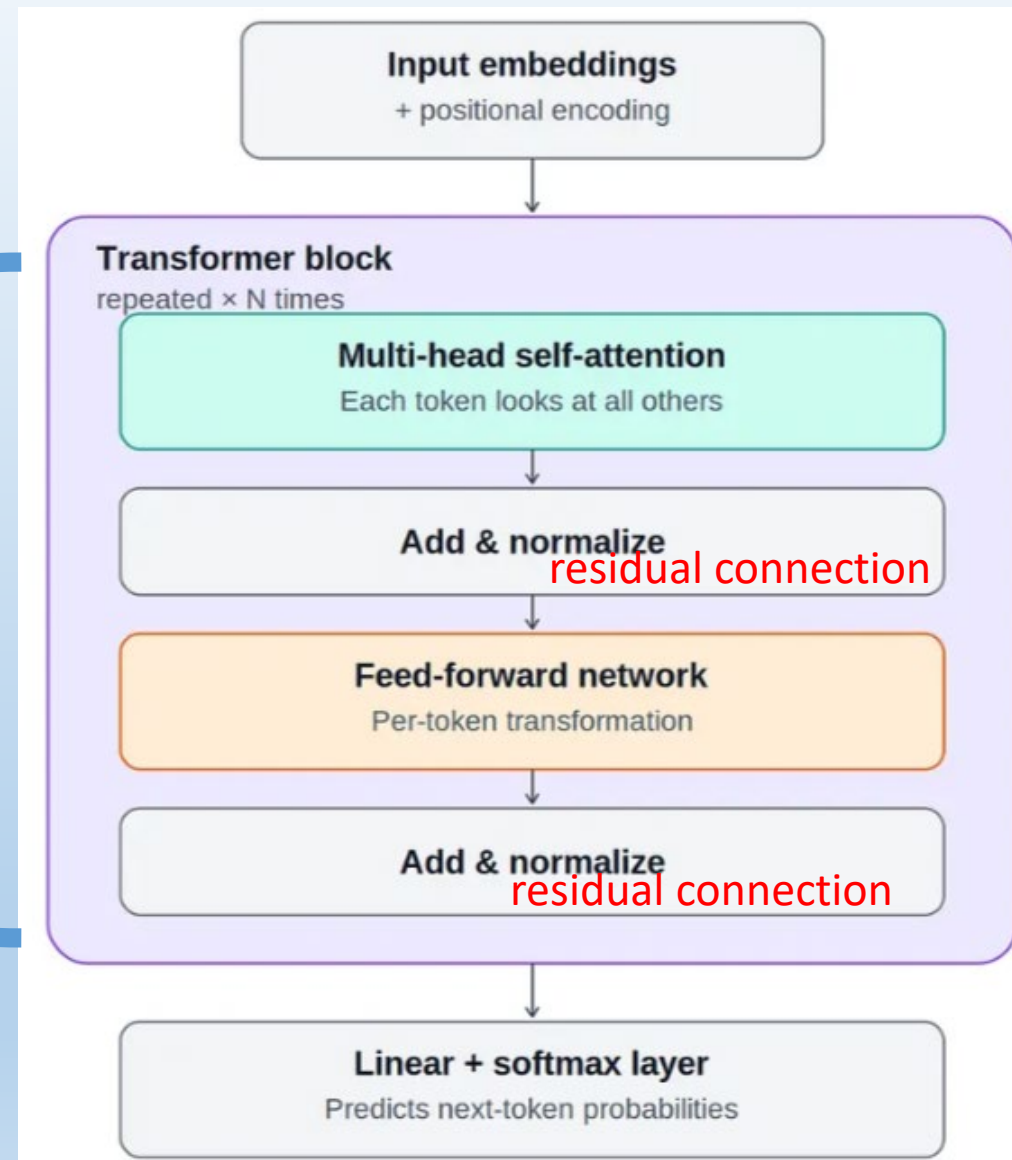
$h = \text{Attention}(X) + X = \text{Output} + X$

**Feedforward Network**

**Residual connection + norm**

residual connection

residual connection



```

# =====
# 11. Full transformer forecasting model
# =====
class TimeSeriesTransformer(nn.Module):
    def __init__(
        self,
        input_size=1,
        d_model=64,
        nhead=4,
        num_layers=2,
        dim_feedforward=128,
        dropout=0.1,
        max_len=500
    ):
        super().__init__()

        self.input_projection = nn.Linear(input_size, d_model)
        self.positional_encoding = PositionalEncoding(d_model, max_len=max_len)

        self.layers = nn.ModuleList([
            TransformerBlock(
                d_model=d_model,
                nhead=nhead,
                dim_feedforward=dim_feedforward,
                dropout=dropout
            )
            for _ in range(num_layers)
        ])

        self.output_head = nn.Linear(d_model, 1)

```

```
def generate_causal_mask(self, seq_len, device):
    # shape: (1, 1, seq_len, seq_len)
    mask = torch.tril(torch.ones(seq_len, seq_len, device=device)).unsqueeze(0).unsqueeze(0)
    return mask
```

```
def forward(self, x, return_attention=False):
    # x: (batch, seq_len, 1)
    x = self.input_projection(x)
    x = self.positional_encoding(x)

    seq_len = x.size(1)
    mask = self.generate_causal_mask(seq_len, x.device)

    all_attention = []

    for layer in self.layers:
        x, attn_weights = layer(x, mask=mask)
        all_attention.append(attn_weights)

    # use final time step representation for next-step forecasting
    last_token = x[:, -1, :]
    out = self.output_head(last_token)

    if return_attention:
        return out, all_attention
    return out
```

A causal mask is applied so each time step cannot attend to future time steps. A causal lower-triangular mask is the standard way to restrict attention to current and past positions in autoregressive forecasting.

$$y(\text{ValuePred}) = \underset{(1 \times 64)}{X_{t_{\text{attention}}}} \cdot \underset{(64 \times 1)}{W_{\text{out}}} + \underset{(1 \times 1)}{b_{\text{out}}}$$

```
# =====  
# 12. Build model and display parameter counts  
# =====  
model = TimeSeriesTransformer(  
    input_size=1,  
    d_model=D_MODEL,  
    nhead=NHEAD,  
    num_layers=NUM_LAYERS,  
    dim_feedforward=DIM_FEEDFORWARD,  
    dropout=DROPOUT,  
    max_len=LOOKBACK  
) .to(device)  
  
print(model)  
print("\nParameter counts")  
print("Total model parameters:", count_parameters(model))  
print("Input projection parameters:", count_parameters(model.input_projection))  
print("Positional encoding parameters:", count_parameters(model.positional_encoding))  
for i, layer in enumerate(model.layers):  
    print(f"Transformer block {i} parameters:", count_parameters(layer))  
    print(f"  Self-attention parameters:", count_parameters(layer.self_attn))  
    print(f"  Feedforward parameters   :", count_parameters(layer.ffn))  
print("Output head parameters:", count_parameters(model.output_head))
```

```

... TimeSeriesTransformer(
  (input_projection): Linear(in_features=1, out_features=64, bias=True)
  (positional_encoding): PositionalEncoding()
  (layers): ModuleList(
    (0-1): 2 x TransformerBlock(
      (self_attn): MultiHeadSelfAttention(
        (W_q): Linear(in_features=64, out_features=64, bias=True)
        (W_k): Linear(in_features=64, out_features=64, bias=True)
        (W_v): Linear(in_features=64, out_features=64, bias=True)
        (W_o): Linear(in_features=64, out_features=64, bias=True)
        (attention): ScaledDotProductAttention(
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
      (norm1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
      (dropout1): Dropout(p=0.1, inplace=False)
      (ffn): PositionwiseFeedForward(
        (net): Sequential(
          (0): Linear(in_features=64, out_features=128, bias=True)
          (1): ReLU()
          (2): Dropout(p=0.1, inplace=False)
          (3): Linear(in_features=128, out_features=64, bias=True)
        )
      )
      (norm2): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
      (dropout2): Dropout(p=0.1, inplace=False)
    )
  )
  (output_head): Linear(in_features=64, out_features=1, bias=True)
)

```

input embeddings  $[64+64=128]$

positional encoding  $[0]$

two transformer blocks

Each  $[64 \times 64 + 64 = 4160]$

$[4160 \times 4 = 16640]$

$[64 + 64 = 128]$

$[64 \times 128 + 128 = 8320]$

$[128 \times 64 + 64 = 8256]$

$[64 + 64 = 128]$

33472

$[64 + 1 = 65]$  one output

## Parameter counts

Total model parameters: 67137 = 128 + 33472 + 33472 + 65

Input projection parameters: 128

Positional encoding parameters: 0

Transformer block 0 parameters: 33472 = 128 + 16640 + 16576 + 128

Self-attention parameters: 16640

Feedforward parameters : 16576

Transformer block 1 parameters: 33472

Self-attention parameters: 16640

Feedforward parameters : 16576

Output head parameters: 65

```

# =====
# 13. Train model
# =====

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

train_losses = []
test_losses = []

for epoch in range(NUM_EPOCHS):
    model.train()
    running_train_loss = 0.0

    for xb, yb in train_loader:
        xb = xb.to(device)
        yb = yb.to(device)

        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()

        running_train_loss += loss.item() * xb.size(0)

    epoch_train_loss = running_train_loss / len(train_loader.dataset)
    train_losses.append(epoch_train_loss)

    model.eval()
    running_test_loss = 0.0

```

```

with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(device)
        yb = yb.to(device)

        pred = model(xb)
        loss = criterion(pred, yb)
        running_test_loss += loss.item() * xb.size(0)

epoch_test_loss = running_test_loss / len(test_loader.dataset)
test_losses.append(epoch_test_loss)

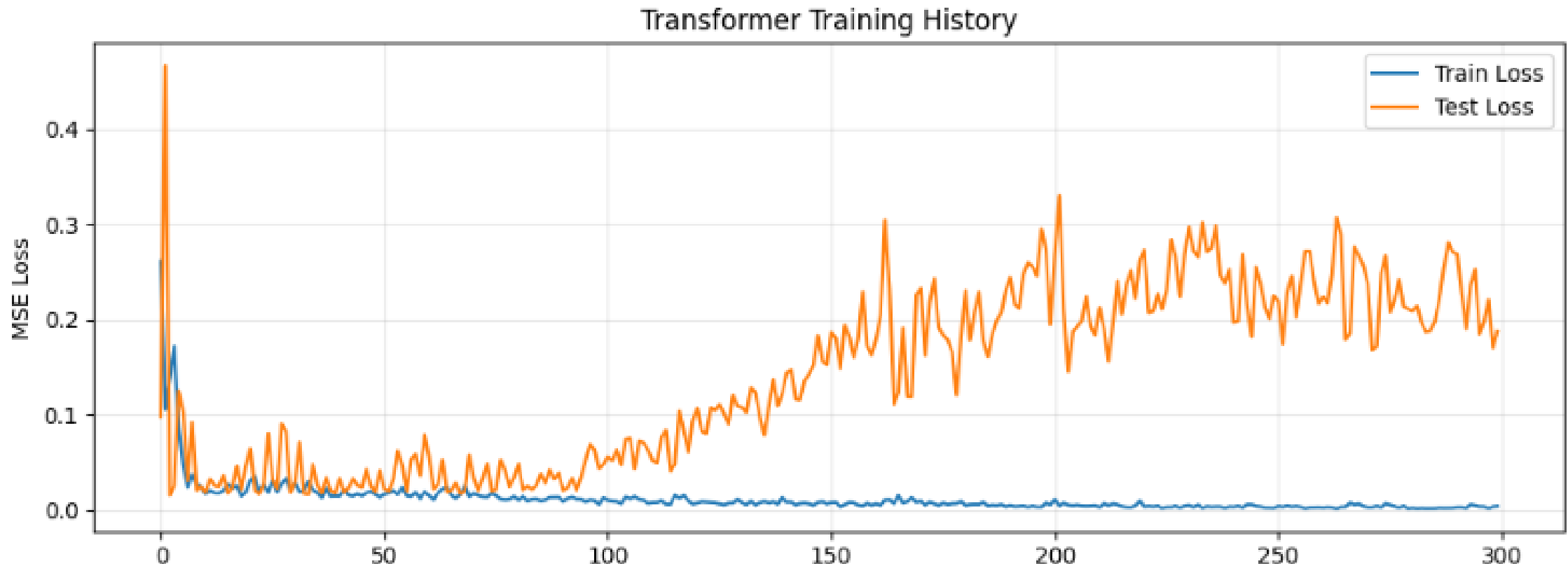
```

|           |                      |                     |
|-----------|----------------------|---------------------|
| Epoch 025 | Train Loss: 0.018958 | Test Loss: 0.080725 |
| Epoch 050 | Train Loss: 0.013838 | Test Loss: 0.041535 |
| Epoch 075 | Train Loss: 0.018193 | Test Loss: 0.020166 |
| Epoch 100 | Train Loss: 0.013397 | Test Loss: 0.048437 |
| Epoch 125 | Train Loss: 0.008084 | Test Loss: 0.104766 |
| Epoch 150 | Train Loss: 0.007081 | Test Loss: 0.152421 |
| Epoch 175 | Train Loss: 0.005244 | Test Loss: 0.191724 |
| Epoch 200 | Train Loss: 0.006120 | Test Loss: 0.194657 |
| Epoch 225 | Train Loss: 0.002318 | Test Loss: 0.210778 |

```
# =====  
# 14. Plot loss curves
```

```
# =====
```

```
plt.figure(figsize=(10, 4))  
plt.plot(train_losses, label="Train Loss")  
plt.plot(test_losses, label="Test Loss")  
plt.title("Transformer Training History")  
plt.xlabel("Epoch")  
plt.ylabel("MSE Loss")  
plt.legend()  
plt.grid(True, alpha=0.3)  
plt.tight_layout()  
plt.show()
```



```

# =====
# 15. One-step prediction on test set
# =====

model.eval()
preds_scaled = []
targets_scaled = []

with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(device)
        pred = model(xb).cpu().numpy()

        preds_scaled.append(pred)
        targets_scaled.append(yb.numpy())

preds_scaled = np.concatenate(preds_scaled, axis=0)
targets_scaled = np.concatenate(targets_scaled, axis=0)

preds = scaler.inverse_transform(preds_scaled).flatten()
targets = scaler.inverse_transform(targets_scaled).flatten()

rmse = np.sqrt(mean_squared_error(targets, preds))
mae = mean_absolute_error(targets, preds)

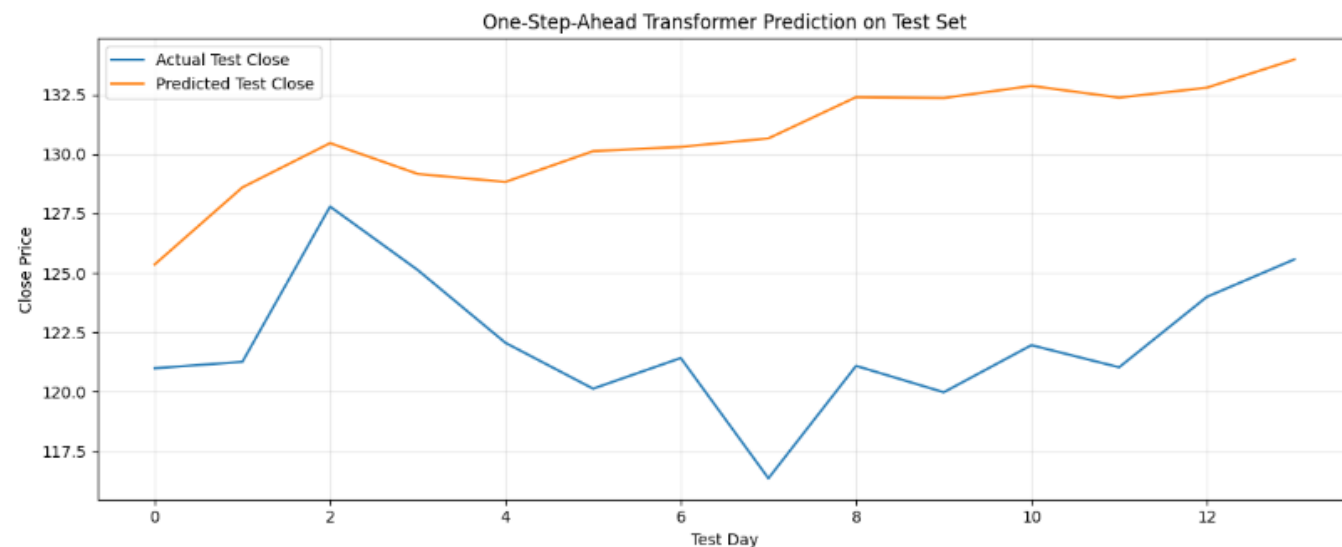
print(f"Test RMSE: {rmse:.4f}")
print(f"Test MAE : {mae:.4f}")

```

```

*** Test RMSE: 9.2650
    Test MAE : 8.6769

```



```

# =====
# 16. Recursive multi-step forecasting
# =====

def recursive_forecast(model, last_window, future_steps, device):
    model.eval()
    window = last_window.copy()
    future_preds_scaled = []

    for _ in range(future_steps):
        x = torch.tensor(window, dtype=torch.float32).reshape(1, len(window), 1).to(device)

        with torch.no_grad():
            next_pred = model(x).item()

        future_preds_scaled.append(next_pred)
        window = np.append(window[1:], next_pred)

    return np.array(future_preds_scaled, dtype=np.float32)

future_steps = 10

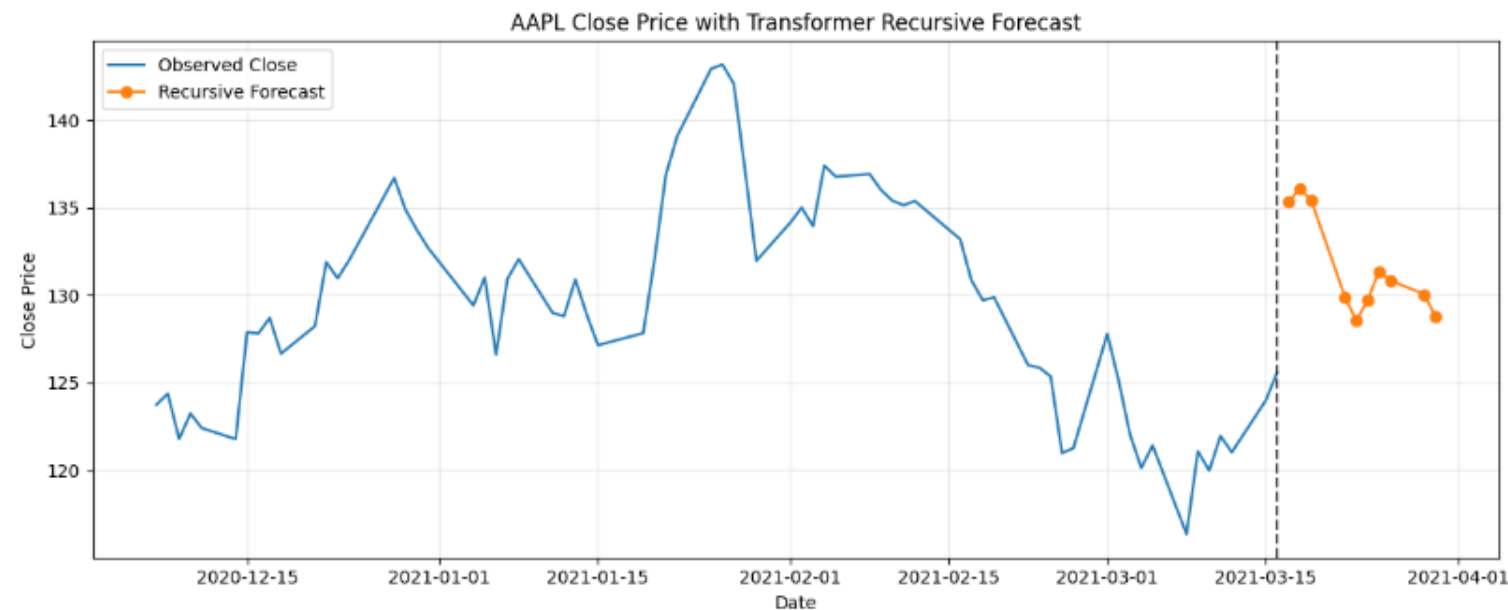
full_scaled = scaler.transform(series.reshape(-1, 1)).flatten()
last_window = full_scaled[-LOOKBACK:]

future_scaled = recursive_forecast(
    model=model,
    last_window=last_window,
    future_steps=future_steps,
    device=device
)

future_prices = scaler.inverse_transform(future_scaled.reshape(-1, 1)).flatten()

print("\nRecursive forecast for next", future_steps, "business days:")
for i, price in enumerate(future_prices, start=1):
    print(f"Day +{i}: {price:.2f}")

```



**Input embeddings** Number of parameters is  $[64+64=128]$

In this univariate time-series Transformer, each input at a time step is initially **one real number**, such as one normalized closing price. The input projection converts that scalar into a 64-dimensional vector before attention is applied.

**1. Input shape** With a 20-day lookback window:

$$X \in \mathbb{R}^{B \times T \times F}$$

where:

B: batch size

T=20: sequence length

F=1: number of input features

For one sample:  $[x_1, x_2, \dots, x_{20}]$

Each  $x_t$  is one real-valued normalized stock price.

$x_1 = 0.42$

$x_2 = 0.57 \dots$

$x_{20} = 0.61$

The model does not initially receive a 64-dimensional vector at each time step. It receives one scalar and then projects that scalar into the model dimension.

## 2. Input projection to D\_MODEL = 64

The code uses: `self.input_projection = nn.Linear(input_size=1, out_features=64)`

parameters [64+64=128]

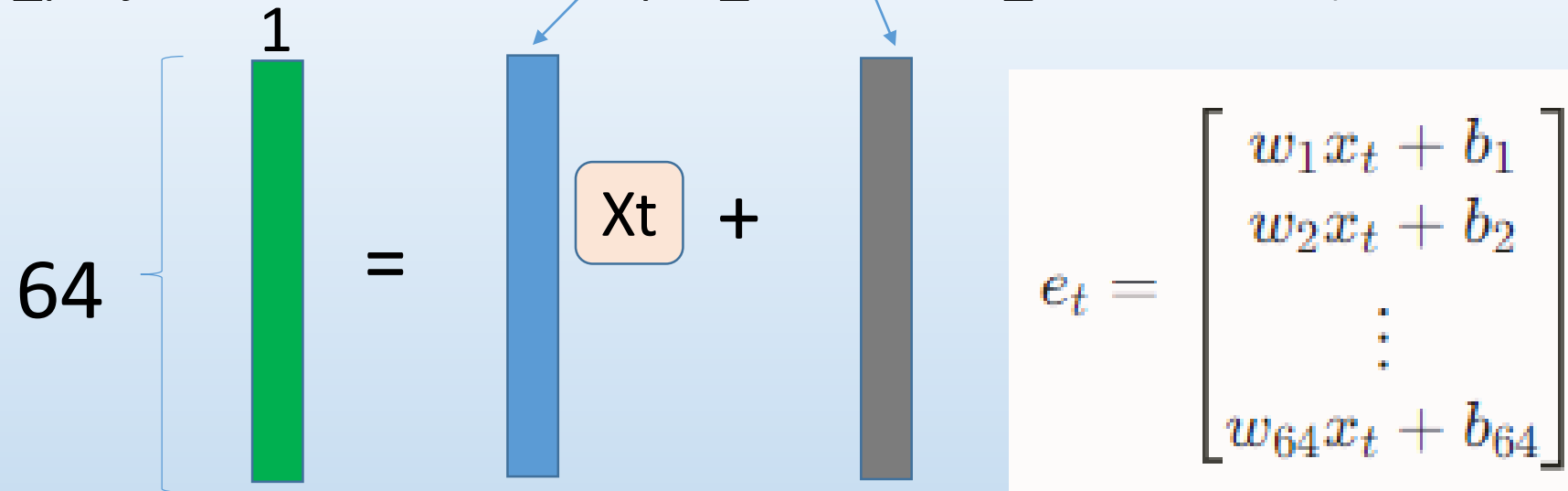
$$e_t = W_e x_t + b_e$$

$$e_t \in \mathbb{R}^{64}$$

$$x_t \in \mathbb{R}^1$$

$$W_e \in \mathbb{R}^{64 \times 1}$$

$$b_e \in \mathbb{R}^{64}$$



$$e_t = \begin{bmatrix} w_1 x_t + b_1 \\ w_2 x_t + b_2 \\ \vdots \\ w_{64} x_t + b_{64} \end{bmatrix}$$

$$e_t = W_e x_t + b_e$$

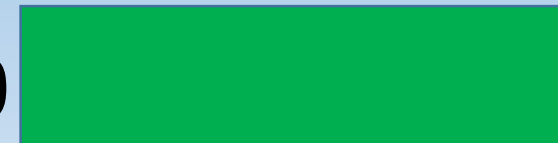
For a batch:

$$X \in \mathbb{R}^{B \times 20 \times 1}$$

the projection produces:

$$E \in \mathbb{R}^{B \times 20 \times 64}$$

20



64

The 64 values are learned features derived from the one input price. They are not 64 additional observed stock variables. They are a learned representation of the scalar input. Thus, even though the original input is one number, the Transformer operates on a 64-dimensional representation.

For a single scalar  $x_t$ , the 64-dimensional embedding is:

### 3. Why use 64 dimensions?

$D\_MODEL = 64$  is the internal feature dimension of the transformer. The final prediction (output) is one value.

The code: `self.output_head = nn.Linear(64, 1)`

$$e_t = \begin{bmatrix} w_1 x_t + b_1 \\ w_2 x_t + b_2 \\ \vdots \\ w_{64} x_t + b_{64} \end{bmatrix}$$

parameters  $[64+64=128]$

$$\hat{y} = W_y h_{20} + b_y$$

$(1 \times 64) + (1 \times 1)$

$(64 \times 1)$

parameters  $[64+1=65]$

## 4. Positional encoding

After input projection, positional encoding is added:

$$z_t = e_t + p_t$$

$$e_t \in \mathbb{R}^{64}$$

$$p_t \in \mathbb{R}^{64}$$

$$z_t \in \mathbb{R}^{64}$$

For all 20 positions:

projected input: (B, 20, 64)

positional encoding: (1, 20, 64)

result: (B, 20, 64)

The Transformer needs positional information because self-attention alone does not inherently know that one value occurred before another.

## 5. Multi-head attention dimensions

$D_{\text{MODEL}} = 64$

$N_{\text{HEAD}} = 4$

Attention mechanism (matching Q and K) are carried out in a lower dimension than the original 64-dim ( $D_{\text{MODEL}}$ ).

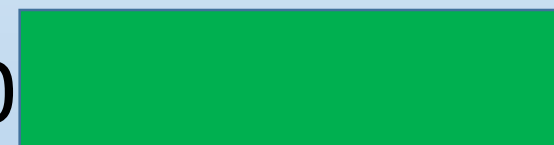
Each attention head has dimension:

$$d_k = \frac{D_{\text{MODEL}}}{N_{\text{HEAD}}} = \frac{64}{4} = 16$$

The input to multi-head attention is:

$$X \in \mathbb{R}^{B \times 20 \times 64}$$

20



64

The query, key, and value projections initially produce 64 features each:

Q before splitting: (B, 20, 64)

K before splitting: (B, 20, 64)

V before splitting: (B, 20, 64)

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

(20x64) (64x64)

**To be exact:**

The number of parameters are  $(64 \times 64) + (1 \times 64)$

$$Q = XW_q^T + b_q$$

Each  $[64 \times 64 + 64 = 4160]$

## 5. Multi-head attention dimensions (cont'd)

Then they are reshaped into four heads:

Q: (B, 4, 20, 16)

K: (B, 4, 20, 16)

V: (B, 4, 20, 16)

The 64 output dimensions are then divided across four heads:

Conceptually,

$$X \left[ W_q^{(1)}, W_q^{(2)}, W_q^{(3)}, W_q^{(4)} \right] + b_q$$

(20x64)[64x16 | 64x16 | 64x16 | 64x16] + bias

.....

$$Q = \left[ \begin{array}{c|c|c|c} Q1 & Q2 & Q3 & Q4 \end{array} \right]$$

(20x16) each

```

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model=64, nhead=4, dropout=0.1):
        super().__init__()

        assert d_model % nhead == 0

        self.d_model = d_model
        self.nhead = nhead
        self.d_k = d_model // nhead

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        # x shape: (B, T, D_MODEL)
        batch_size, seq_len, _ = x.shape

        Q = self.W_q(x)
        K = self.W_k(x)
        V = self.W_v(x)

```

```

# (B, T, 64) -> (B, T, 4, 16)
Q = Q.view(
    batch_size,
    seq_len,
    self.nhead,
    self.d_k
)

```

```

# (B, T, 4, 16) -> (B, 4, T, 16)
Q = Q.transpose(1, 2)

```

Q after splitting: (16, 4, 20, 16)

Similar things for K and V.

## 9. Scaled dot-product attention dimensions

For one head:

$$Q = [ \textcolor{red}{Q1} \mid Q2 \mid Q3 \mid Q4 ]$$

(20x16) each

$$K = [ \textcolor{red}{K1} \mid K2 \mid K3 \mid K4 ]$$

(20x16) each

$$V = [ \textcolor{red}{V1} \mid V2 \mid V3 \mid V4 ]$$

(20x16) each

$$\text{score} = \frac{Q_1 \cdot K_1^T}{\sqrt{16}} \quad \text{Dimension of score is (20x20)}$$

Attention weights = software(score)

$$\text{Head1} = \text{Attention weights} \cdot [ \textcolor{red}{V1} ] \quad \text{is output from Head1}$$

(20x16)                  (20x20)                  (20x16)

20 [ head1, head2, head3, head 4 ] will be of dimension (20x64)

64

Concatenated attention output

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

(20 x 64)    same as input                  (20 x 64 )                  (64 x 64 )

$$\begin{matrix}
 Q_1 & \cdot & K_1^T & = & 
 \begin{bmatrix}
 -q_1-- \\
 -q_2-- \\
 -q_3-- \\
 \dots \\
 \dots \\
 -q_{20}--
 \end{bmatrix}
 \begin{bmatrix}
 k_1^T & k_2^T & \dots & k_{20}^T
 \end{bmatrix} \\
 (20 \times 16) & (16 \times 20) & & & 
 \end{matrix}$$

**masking**



$$Q_1 \cdot K_1^T = \begin{bmatrix}
 q_1 \cdot k_1^T & q_1 \cdot k_2^T & \dots & \dots & q_1 \cdot k_{20}^T \\
 q_2 \cdot k_1^T & q_2 \cdot k_2^T & \dots & \dots & q_2 \cdot k_{20}^T \\
 \dots & \dots & \dots & \dots & \dots \\
 q_{20} \cdot k_1^T & q_{20} \cdot k_2^T & \dots & \dots & \dots
 \end{bmatrix}$$

softmax (attention matrix) is (20 x 20) and the first row is how the others response to the first query. After softmax(), the sum of each row is 1.

D\_MODEL = 64

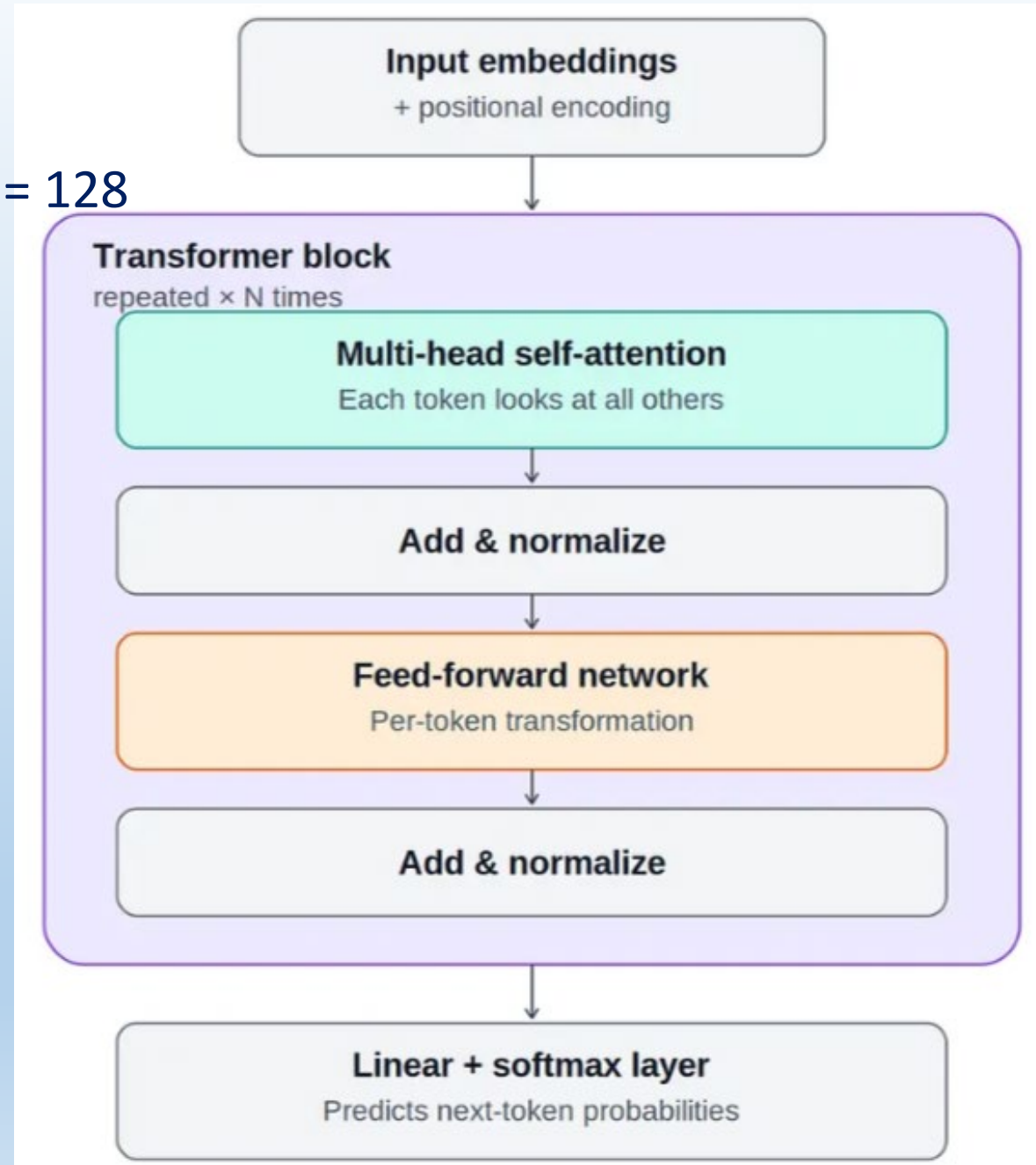
NHEAD = 4

NUM\_LAYERS = 2

DIM\_FEEDFORWARD = 128

LOOKBACK = 20

INPUT\_SIZE = 1



(20x64)

Q,K,V are (64x64)  
dot-product vector (1x16)  
Attention matrix is (20x20)

64  $\rightarrow$  128  $\rightarrow$  64

## 10. Batch processing

In the PyTorch code, **batch** means the number of training examples processed together in one forward and backward pass. It is not a batch file or a special Transformer component.

### Example with stock prices

Suppose the previous 20 days are used to predict the next day:

$$X_i = [p_i, p_{i+1}, \dots, p_{i+19}]$$

$$y_i = p_{i+20}$$

From a long price series, many overlapping training examples are created:

Sample 1: days 1–20 → predict day 21

Sample 2: days 2–21 → predict day 22

Sample 3: days 3–22 → predict day 23

If `BATCH_SIZE = 16`, then PyTorch processes 16 such 20-day samples together !

`x.shape = (16, 20, 1)`

This means:

- 16 independent training windows
- 20 days in each window
- 1 feature per day: the closing price

After input projection:

$(16, 20, 1) \rightarrow (16, 20, 64)$

The 64 is the Transformer representation dimension, not the batch size.

Pytorch code:

```
train_loader = DataLoader(  
    TensorDataset(X_train, y_train),  
    batch_size=16, shuffle=True )
```

takes the complete training dataset and returns it in groups of 16 samples.

## Why use batches?

**1. Memory efficiency:** Example Entire dataset: 1,000 samples ; Batch size: 16 samples

The model processes:

Batch 1: samples 1–16

Batch 2: samples 17–32 ...

## 2. Faster computation

Modern CPUs and GPUs are designed to perform operations on many values simultaneously. Processing 16 sequences together is usually faster than processing 16 sequences one at a time.

## 3. More stable gradient estimates

The model calculates the loss over the batch. The gradients are then calculated from the average batch loss. The use of more samples gives a more stable gradient than using only one sample.

**Note that batch size does not change the model architecture. The batch dimension is not part of the trainable weight dimensions. The batch allows multiple independent 20-day stock-price windows to be processed together. It improves computational efficiency, reduces memory requirements compared with processing the entire dataset at once, and provides a more stable average gradient for training.**

**The End**